

Forensics Case Report: Insider Timestomping & MFT Analysis

CASE ID: 006 - Operation Time Warp

EXAMINER: Maxwell Bosiako Antwi

DATE OF EXAMINATION: May 2026

TARGET SYSTEM: Corporate Engineering Workstation

1. Executive Summary

An investigation was launched into a suspected insider threat on a corporate engineering workstation. The employee was suspected of downloading a malicious network utility (`scanner.exe`) and utilizing anti-forensic timestomping techniques to backdate the file to 2021, attempting to blend it in with legacy system files.

Through live acquisition and deep parsing of the Master File Table (`$MFT`) and Update Sequence Number Journal (`$UsnJrnl`), the investigation successfully bypassed the manipulated user-space metadata. The analysis proved definitively that the file was introduced on June 5, 2026, directly contradicting the manipulated `$STANDARD_INFORMATION` timestamps and confirming deliberate anti-forensic activity.

2. Case Background & Scope

A corporate insider allegedly downloaded `scanner.exe` to a secure engineering workstation. To evade standard auditing and timeline analysis, the suspect used Windows API manipulation to alter the file's metadata.

Objective:

1. **Simulation:** Replicate the threat actor's methodology by deploying and timestomping a target file via PowerShell.
2. **Investigation:** Perform raw NTFS artifact analysis to expose the temporal manipulation without relying on the compromised Windows Explorer graphical interface.

3. Evidence Acquisition

Because the `$MFT` is locked by the active Windows operating system, standard file copying methods are ineffective.

- **Acquisition Tool:** AccessData FTK Imager
- **Methodology:** Live logical acquisition. The locked `$MFT` and `$UsnJrnl` system files were extracted directly from the root `C:\` directory and `C:$EXTEND\` respectively.
- **Preparation:** Extracted files were unhidden using the command line (`attrib -h -s .\$MFT`) to prepare for parsing.

4. Methodology & Analysis

Phase 1: Attack Simulation (The Subversion)

To understand the artifact generation, the attack was first simulated. A dummy payload (`C:\ForensicLab\scanner.exe`) was created.

Step 1: Staging the Payload Using PowerShell (Run as Administrator), a mock environment was built to simulate an insider downloading a malicious utility:

- **Directory Creation:** A new directory was created at `C:\ForensicLab\` to house the evidence.

```
PS C:\WINDOWS\system32> New-Item -ItemType Directory -Path C:\Users\Max\Desktop\Forensics Foundation\Lab cases\TimestompLab

Directory: C:\Users\Max\Desktop\Forensics Foundation\Lab cases

Mode                LastWriteTime         Length Name
----                -
d-----          5/28/2026  10:46 AM                TimestompLab

PS C:\WINDOWS\system32>
```

- **File Creation:** The dummy malicious payload, `scanner.exe`, was generated inside this directory.

```
PS C:\WINDOWS\system32> New-Item -ItemType File -Path C:\Users\Max\Desktop\Forensics Foundation\Lab cases\TimestompLab\scanner.exe

Directory: C:\Users\Max\Desktop\Forensics Foundation\Lab cases\TimestompLab

Mode                LastWriteTime         Length Name
----                -
```

- **Baseline Verification:** Before any manipulation occurred, the baseline timestamps written naturally by the OS kernel were checked. This confirmed that the file's initial Creation, Last Write, and Last Access times accurately reflected the true current date (June 5, 2026).

Step 2: Executing the Timestomp With the baseline established, the user-space metadata layer was targeted to simulate the anti-forensics tool.

- **Establishing the Fake Date:** A target date of five years prior was set using the command: `$FakeDate = Get-Date "2021-01-15 09:30:00"`
- **API Manipulation:** The Win32 API was manipulated via PowerShell to forcefully overwrite the file's `$STANDARD_INFORMATION` attributes
 - `CreationTime`
 - `LastWriteTime`
 - `LastAccessTime`

```

PS C:\WINDOWS\system32> $FakeDate = Get-Date "2021-01-15 09:30:00"
PS C:\WINDOWS\system32> $TargetFile = Get-Item C:\Users\Max\Desktop\Forensics Foundation\Lab cases\TimestampLab\scanner.exe
PS C:\WINDOWS\system32> $TargetFile.CreationTime = $FakeDate
PS C:\WINDOWS\system32> $TargetFile.LastWriteTime = $FakeDate
>>
PS C:\WINDOWS\system32> $TargetFile.LastAccessTime = $FakeDate
PS C:\WINDOWS\system32>

```

All three values were artificially rolled back to **January 15, 2021, 09:30:00**. Viewed through standard system tools (like Windows File Explorer), the file appeared to be five years old.

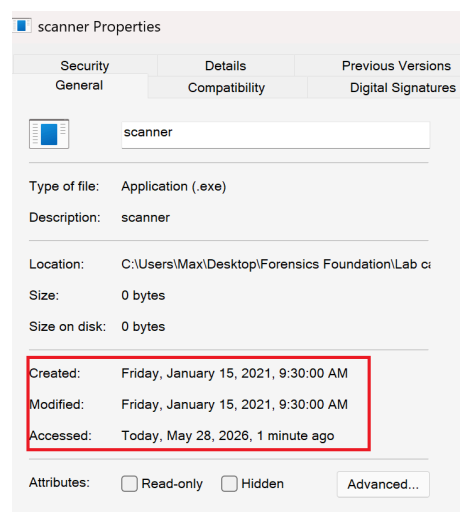
- **Verification:** The faked timeline was then verified through standard system views (e.g., Windows Explorer GUI). To casual observation or basic auditing, the file successfully appeared to have been created in January 2021.

```

PS C:\WINDOWS\system32> Get-Item C:\Users\Max\Desktop\Forensics Foundation\Lab cases\TimestampLab\scanner.exe | Select-Object Name, CreationTime, LastWriteTime

```

Name	CreationTime	LastWriteTime
scanner.exe	1/15/2021 9:30:00 AM	1/15/2021 9:30:00 AM



Phase 2: Forensic Parsing (The Investigation)

Step 1: Dumping Live System Artifacts: Because the \$MFT is locked by the active Windows operating system, standard copying is impossible. Live extraction was performed using AccessData FTK Imager:

1. **Mounting the Drive:** FTK Imager was launched as Administrator. The evidence item was added via **File** → **Add All Attached Devices** or **Add Evidence Item** → **Logical Drive**, selecting the **C:** drive.
2. **Exporting the \$MFT:** Navigating to the root of the **C:** drive in the evidence tree, the hidden system file **\$MFT** was located at the top of the file list under **\$**. It was exported directly to a local desktop working directory.

3. **Preparing the Evidence:** Because the exported **\$MFT** retains its OS-level hidden attributes, PowerShell was used to unhide the file for parsing: **attrib -h -s .\ \$MFT** to strip the hidden (h) and system (s) flag off the file.

Step 2: Parsing the Exported File with MFTECmd With the **\$MFT** extracted, Eric Zimmerman's MFTECmd was utilized via the command prompt to parse the raw hex data into a human-readable format.

- **Command Executed:** **MFTECmd.exe -f <filepath of \$MFT> --csv <path to where you want to save the csv>** This generated **MFTECmd_Output.csv** in the **C:\Users\H4ck3r\Desktop\MFTECmd** directory.

```

Windows [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

Select Administrator: Windows PowerShell
'ls' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\H4ck3r\Desktop\MFTECmd>.MFTECmd.exe -f "C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT>" --csv "C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT>"
MFTECmd version 2026.5.0

Author: Eric Zimmerman (saericzimmerman@gmail.com)
https://github.com/EricZimmerman/MFTECmd

Command line: -f C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT> --csv C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT>

File type: MFT

Processed C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT> in 9.6034 seconds

C:\Users\H4ck3r\Desktop\TimeStompLab\<MFT>: FILE records found: 172,938 (Free records: 332) File size: 169.2MB
CSV output will be saved to C:\Users\H4ck3r\Desktop\TimeStompLab\20260604173622_MFTECmd_<MFT>_Output.csv

C:\Users\H4ck3r\Desktop\MFTECmd>

```

Step 3: Extracting the Evidence: The generated CSV output was opened in a spreadsheet editor. Using **Ctrl + F**, the target file **scanner.exe** was located to examine its raw kernel-level timestamp headers.

	A	B	C	D	E	F	G
1	FileName	Created0x10	Created0x30	LastModified0x10	LastModified0x30	LastRecordChange0x10	LastRecordChange0x30
2	\$MFT	5/28/2026 19:55		55:02.1		55:02.1	
3	\$MFTMirr	55:02.1		55:02.1		55:02.1	
4	\$LogFile	55:02.1		55:02.1		55:02.1	
5	\$Volume	55:02.1		55:02.1		55:02.1	
6	\$AttrDef	55:02.1		55:02.1		55:02.1	
7	.	5/28/2026 19:55		6/4/2026 16:21	5/28/2026 19:55	6/4/2026 16:21	
8	\$Bitmap	5/28/2026 19:55		5/28/2026 19:55		5/28/2026 19:55	
9	scanner.exe	1/15/2021 9:30	6/5/2026 12:59	1/15/2021 9:30	5/28/2026 12:59	6/5/2026 13:08	6/5/2026 13:08
10	\$Bitmap:\$SRA	5/28/2026 19:55		5/28/2026 19:55		5/28/2026 19:55	
11	\$Boot	55:02.1		55:02.1		55:02.1	
12	\$BadClus	55:02.1		55:02.1		55:02.1	
13	\$BadClus:\$Bad	55:02.1		55:02.1		55:02.1	
14	\$Secure	55:02.1		55:02.1		55:02.1	

- **STANDARD_INFORMATION(\$SI) Clock (0x10):** The **Created0x10** and **LastModified0x10** columns reflected the manipulated date of **January 15, 2021**. This confirmed the scope of the timestamp attack.

- **FILE_NAME(FN) Clock (0x30):** The **Created0x30** and **LastModified0x30** columns successfully resisted the Win32 API manipulation, displaying the true creation date of **June 5, 2026**.
- **MFT Entry Modified (LastRecordChange0x10):** Interestingly, this timestamp remained unchanged, showing **June 5, 2026**. Because this record is directly controlled by the system clock and updates whenever the MFT record structure itself is modified (e.g., when the attacker altered the \$SI timestamps), it served as immediate proof of recent tampering.

5 Key Findings

5.1. MFT Timestamp Discrepancy (The 0x10 vs. 0x30 Conflict)

Analysis of the **\$MFT** record for **scanner.exe** revealed a classic timestamp mismatch, confirming deliberate manipulation. NTFS stores timestamps in two primary attributes, which reacted differently to the PowerShell manipulation:

Time Attribute	Reported Date/Time	Forensic Significance
\$STANDARD_INFORMATION (0x10)	Jan 15, 2021	Compromised. This is the user-level attribute modified by the timestamp attack.
\$FILE_NAME (0x30)	June 5, 2026	True Date. This attribute is only modified by the system kernel (e.g., when a file is renamed or moved). It successfully retained the actual creation date.
Entry Modified (LastRecordChange)	June 5, 2026	True Date. This tracks when the MFT record <i>itself</i> was changed. It updated automatically when the attacker altered the 0x10 attribute, betraying the recent activity.

5.2. USN Journal Corroboration (*UsnJrnl*:J)

To definitively prove the sequence of events and corroborate the June 5th timestamp, the Update Sequence Number Journal (**\$UsnJrnl**) was exported from **C:\$EXTEND\$UsnJrnl** and parsed.

Because the USN Journal acts as an immutable, chronological ledger of file system changes, it captured the attacker's exact sequence of actions in real-time. Filtering the parsed CSV for `scanner.exe` revealed the following chronological event chain on June 5, 2026:

- **Event 1: Payload Staging (USN_REASON_FILE_CREATE)** The journal explicitly recorded the creation of `scanner.exe`. This definitively proves the file was introduced to the disk on June 5, 2026, completely invalidating the 2021 `$STANDARD_INFORMATION` timestamp.
- **Event 2: Payload Writing (USN_REASON_DATA_EXTEND / DATA_TRUNCATION)** Immediately following creation, the journal logged the data being written to the file as the payload was downloaded/copied to the directory.
- **Event 3: The Timestamp Execution (USN_REASON_BASIC_INFO_CHANGE)** Crucially, the journal logged a `BASIC_INFO_CHANGE` event. In NTFS forensics, this specific update reason is the primary indicator of timestomping, as it is triggered when a user or script forcefully alters the metadata (Creation/Modification/Access times) in the `$STANDARD_INFORMATION` attribute.
- **Event 4: Operation Complete (USN_REASON_CLOSE)** The file handle was closed, cementing the manipulated metadata onto the disk.

Significance: The `$UsnJrnl` leaves no room for ambiguity. It not only confirms the true creation date found in the MFT `$FILE_NAME` attribute but also documents the exact moment the suspect executed the timestomping script (`BASIC_INFO_CHANGE`), turning a suspected anomaly into proven anti-forensic activity.

6. Conclusion

The presence of `scanner.exe` is not a legacy artifact from 2021. The deep-dive analysis of the NTFS `$MFT` attributes conclusively proves the file was introduced to the system on June 5, 2026.

The severe discrepancy between the `$STANDARD_INFORMATION` (0x10) and `$FILE_NAME` (0x30) timestamps, alongside the recent MFT Entry Modified date, provides definitive proof of intentional timestomping. The insider deliberately attempted to subvert forensic timelines, validating the suspicion of malicious intent.